Mini-Project 1 — GEMM, and a language model that runs on it

You are going to build a fast matrix-multiply (GEMM) engine, and then — as two optional challenges — watch it power a real language model that writes children's stories, faster and faster. The project is in **three parts**, and the first stands completely on its own.

> **The glimpse.** Almost everything a language model does when it generates text is matrix multiply. So "build a good GEMM" and "make a language model fast" are the same task at two zoom levels. **Part 1 is pure GEMM** — no model, nothing to install, just make the math fast. **Parts 2 and 3 are the payoff**: drop your GEMM into an actual storyteller model and measure the tokens per second climb. If the language-model side ever feels like too much, ignore it — Part 1 is the graded core and needs none of it.

The three parts

Part	What it is	Self-contained?	You measure
Part 1 — Pure GEMM	Implement & optimize GEMM for a fixed set of shapes, in **float32 and int8**	✅ just a C compiler	correctness + **GFLOP/s**
Part 2 — Challenge 1	Plug your GEMM into a real SLM (`llama2.c` / TinyStories) and run it	needs the model + a node	**tokens/sec**

Part	What it is	Self-contained?	You measure
Part 3 — Challenge 2	Do it in int8 — 4× less weight memory	needs the model	tokens/sec + memory + story quality

Background reading for Parts 2–3 (no ML assumed): transformer-from-scratch.md — what a language model actually computes, and why it's 43 matrix–vector products per token that become GEMMs when you batch.

Prerequisites

- **Part 1:** a C compiler. That's it. `scripts/synth_cases.sh` makes the test data.
- **Parts 2–3:** the model (`scripts/setup.sh` fetches a 60 MB checkpoint) and a compute node; the hardware run uses the U50 host (your course starter kit has the machine-specific details).

Codebase map

Everything generated (the model clone, weights, binaries, `cases/`) lands in `build/` and `cases/` and is git-ignored. The tracked files are small:

```
labs/tinystories/
├── README.md
├── part1-pure-gemm.md · part2-integrate-slm.md · part3-quantized-slm.md
├── transformer-from-scratch.md          background: how a transformer computes
├── src/
│   ├── student_gemm.h                    ← the float contract (you implement this)
│   ├── student_gemm_ref.c                reference float GEMM (so it all runs today)
│   ├── student_gemm_fpga.cpp             template: same contract, offloaded to your kerr
│   ├── student_gemm_int8.h / _ref.c      ← the int8 contract + reference
│   ├── gemm_test.c / gemm_test_int8.c    offline testers (float / int8)
│   ├── synth_case.c                      makes synthetic GEMM test cases (Part 1)
│   ├── slm_backend.c                     the SLM seam: model's matmul() → student_gemm()
│   └── dump_backend.c                    captures REAL model matmuls to cases/*.bin
└── scripts/
    ├── synth_cases.sh · test_gemm.sh · test_gemm_int8.sh    Part 1: make data, test your
```

```
├── setup.sh · profile.sh                    Parts 2–3: get the model, pr
├── dump_cases.sh                            capture real matmuls for tes
├── build_slm.sh · run_hw.sh                 plug your GEMM into the SLM
```

The whole workflow: you write `student_gemm` (and, for Part 3, `student_gemm_int8`). Test them with `test_gemm.sh` / `test_gemm_int8.sh`; plug them in with `build_slm.sh` / `run_hw.sh`.